

GIT & 자소서

기록을 관리하고, 경험을 설득력 있게 표현하는 법

DK

AI · 데이터 개발자



전체 강의 구성

01

Git 개념과 활용 사례

02

Git 기본 구조와
주요 명령어

03

Git 브랜치·협업
개념과 설치

04

Git 설치와 실습
(별도 HTML 자료)

05

놓친 명령어·브랜치 전략
README·AI 활용까지

06

자소서 문항 이해와
기본 작성법

07

AI 활용 자소서
작성법과 주의사항

08

면접관이 본 문제
사례 및 종합 Q&A

01

Git 개념과 활용 사례

오늘 기억할 세 가지

1

GIT

Git과 GitHub는 다르다!!

도구의 정체성부터 구분

2

기능

기록 · 협업 · 복구

Git이 필요한 세 가지 기능

3

활용

AI 시대에서 GIT은 필수

개발자를 넘어 모든 직무에 적용

Git이 무엇인가요?



처음 Git을 접하면 가장 먼저 떠올리는 답



코드 올리는 곳 아닌가요?

Git은 코드를 올리는 곳이 아닙니다.

Git의 정의

핵심 정의



형상 관리 / 버전 관리 도구

코드나 문서 작업의 변경 이력을 저장하는 도구

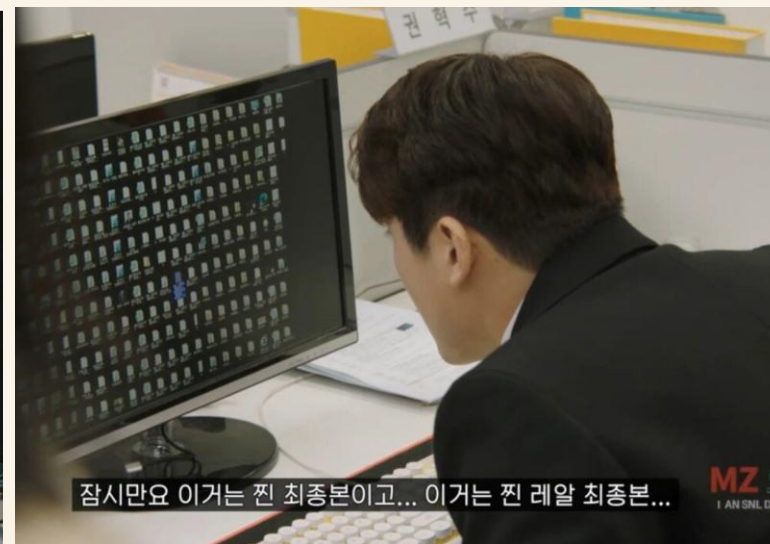
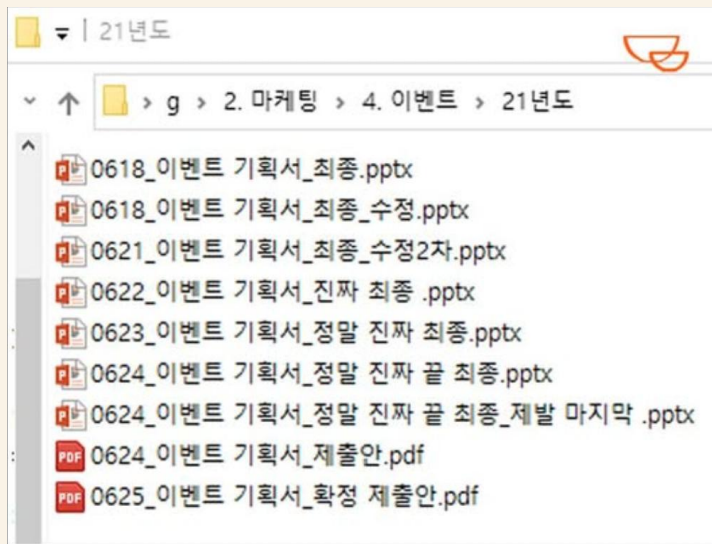
언제

누가

무엇을 수정

형상 관리 시스템, 이 한 문장만 기억할 것!!!

Git이 필요한 이유



버전 관리의 필요성

파일명으로 버전 구분

-  자소서_초안.docx
-  자소서_수정.docx
-  자소서_최종.docx
-  자소서_진짜최종.docx
-  자소서_썬최종_진짜.docx

※ 무엇이 바뀌었는지 파일명만으로는 알 수 없음.

Git으로 관리

 자소서.docx

- v1 최초 작성
- v2 경험 사례 보강
- v3 분량 조정
- v4 최종 검토 반영

※ Git은 파일명이 아니라, 변경 자체를 기록

변경 이력 관리



01

변경 내용

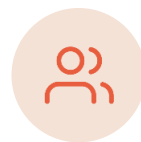
수정한 코드와 문서



02

변경 시점

언제 작업했는지



03

작업 주체

누가 수정했는지

기록이 쌓이면 비교하고, 필요한 시점으로 되돌아갈 수 있음

내 작업 기록



작업 기록 도구

- 수정한 코드·내용의 기록과 이력 추적 가능
- 실수했을 때 되돌리기 가능
- 작업 흐름을 시간순으로 보관

커밋 기록 예시

COMMENT	DATE
덧셈 기능 추가	14 HOURS AGO
뺄셈 기능 추가	12 HOURS AGO
곱셈 기능 추가	10 HOURS AGO
나눗셈 기능 추가	9 HOURS AGO
계산기 최초 버전	6 HOURS AGO

협업 기록

작업 히스토리 예시



PERSON A
덧셈 기능 개발

14시간 전



PERSON B
뺄셈 기능 개발

9시간 전



PERSON C
계산기 화면 개발

3시간 전

협업 도구



여러 사람이 같은 파일을 동시에 고칠 수 있음



누가, 언제, 무엇을 수정했는지 확인 가능



충돌이 나도 해결하고 정리하는 방식 제공

프로그램 버전 관리

공지사항

카카오톡 운영정책 개정 안내	2025.05.16	>
[안내] SKT 유심 정보 유출 관련 카카오톡 보안 조치 및 이용자 유의사항	2025.04.30	>
카카오톡 운영정책 개정 안내 (미성년자 보호조치 관련)	2025.04.23	>
4.3.5 버전 업데이트 안내	2025.02.06	>
4.3.0 버전 업데이트 안내	2024.12.18	>
카카오톡 PC버전 공식 다운로드 설치 경로 안내	2024.12.13	>
4.2.5 버전 업데이트 안내	2024.11.13	>
4.2.0 버전 업데이트 안내	2024.10.07	>



프로그램 버전 관리



프로그램 버전 관리 도구

1

기능 추가, 버그 수정 등 변경 단계를 버전으로 관리

2

릴리즈 버전 이력도 함께 관리 가능

3

필요하면 이전 버전으로 돌아갈 수 있음



릴리즈 노트 예시

v2.3.0

검색 필터 기능 추가

v2.2.1

로그인 오류 수정

v2.2.0

UI 다크모드 지원

v2.1.0

초기 정식 배포

Git과 GitHub



git

Git

버전 관리 도구

기록하고 복구하는 기능을 지원하는 핵심 엔진



GitHub

GitHub

클라우드 협업 플랫폼

Git 기록과 코드를 저장·공유하고, 팀원과 함께 보는 Git 기반 플랫폼

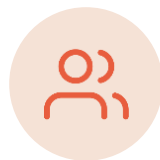
Git을 사용하는 세 가지 이유



01

다양한 플랫폼

GitHub·GitLab·Bitbucket 등 지원



02

넓은 생태계

오픈소스와 기업에서 널리 사용



03

분산형 구조

각자가 전체 저장소 이력을 보유

다양한 업무에서의 Git 활용



문서·기획 관리

기획안·정책·프롬프트 변경 이력



데이터·분석 관리

분석 코드·실험 결과·재현 기준



콘텐츠·운영 관리

문구·설정·자동화 작업 이력



AI 업무 관리

프롬프트·에이전트 설정·지식 문서

AI와 함께 일할수록, 결과보다 '과정'을 기록하는 일이 중요

개발자 중심에서 다양한 업무로

항목	과거의 Git	AI 시대의 Git
주요 사용자	개발자 중심	기획·데이터·디자인·운영 등
관리 대상	소스 코드와 설정 파일	코드·문서·AI 프롬프트·AI 설정
사용 목적	개발 협업과 배포	AI 작업 재현·변경 추적·협업
핵심 질문	누가 코드를 바꿨는가?	어떤 지시와 자료로 결과가 나왔는가?

AI 시대의 Git 핵심 정리

GIT

기록

협업

재현

개발자를 넘어, AI를 활용하는 모든 직무의 기록·협업 도구



AI 업무의 변경 이력을 관리하는 필수 기반

많은 회사가 코드·문서·프롬프트·AI 설정 관리에 Git을 활용합니다.

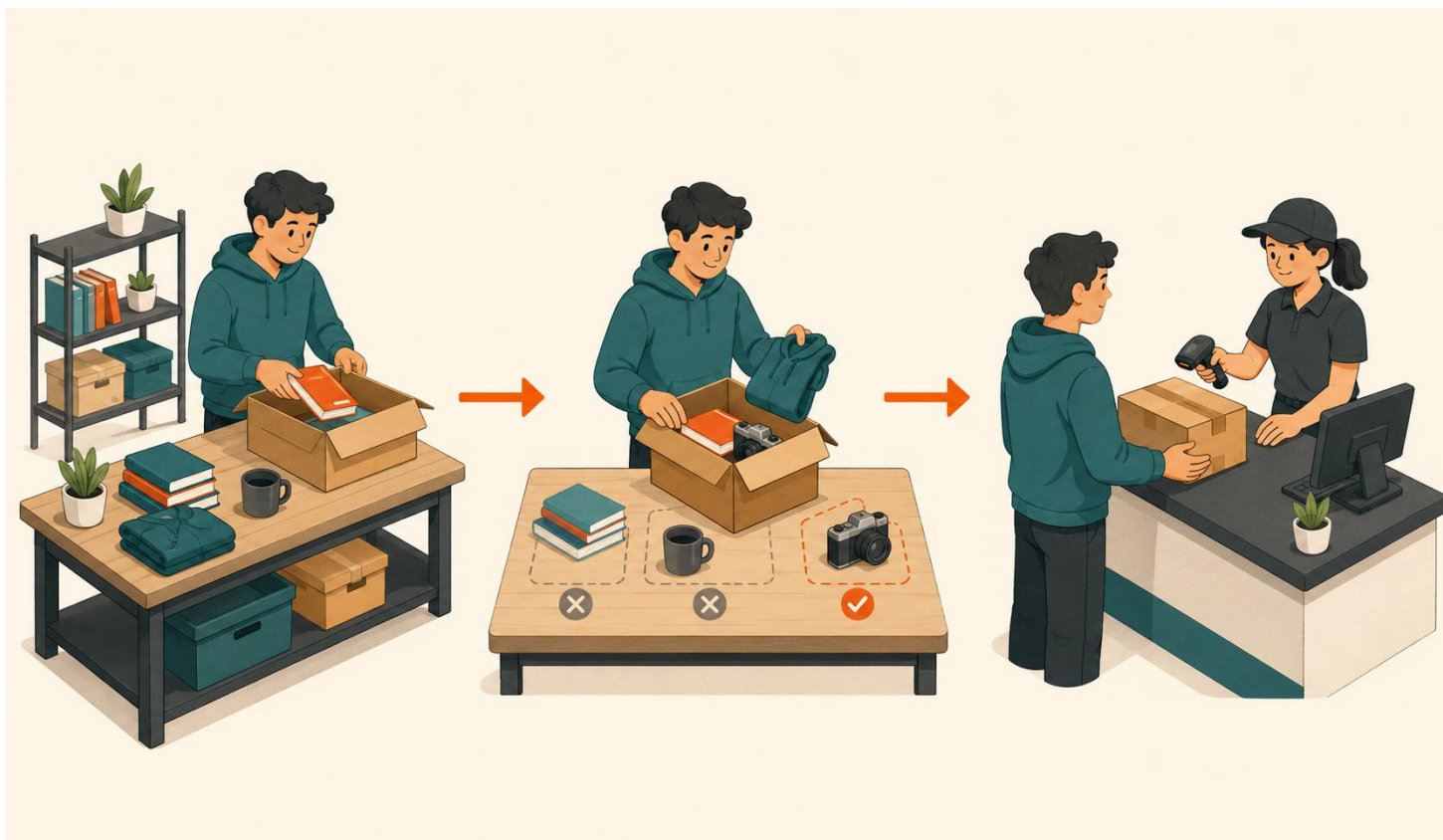
02

Git 구조와 필수 명령어

내 컴퓨터 안에서 변경을 기록하는 세 공간과 핵심 명령어

ANALOGY

택배를 보내는 과정으로 Git 흐름 이해하기



핵심 비유

모든 물건을 바로 보내지 않고, 포장할
것을 고른 뒤 한 번에 접수합니다.

Git은 변경을 세 공간으로 이동시켜 기록

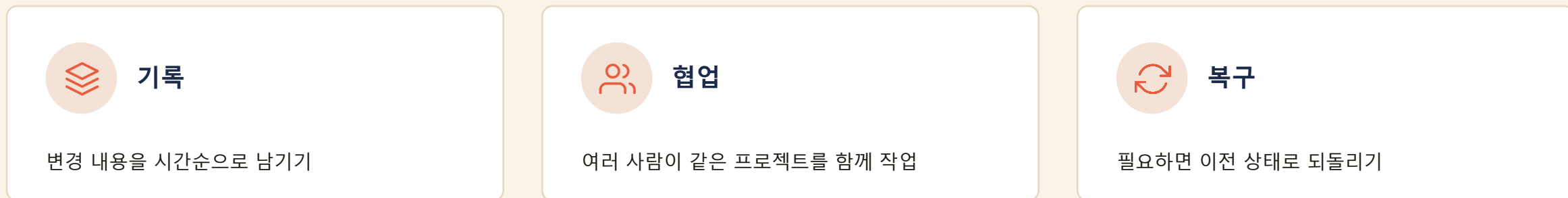


SETUP

이 작업 공간에 Git을 설치하면 시작됩니다



앞서 배운 기록·협업·복구 기능을, 이제 이 폴더 안에서 실제로 사용할 수 있게 됩니다



지금부터는 이 작업 공간에 Git이 이미 설치되어 있다고 가정하고, Working Directory부터 하나씩 살펴보겠습니다

Working Directory는 실제로 물건을 포장하는 내 작업 공간

내 작업 공간에서는

- 새 파일 만들기
- 내용 수정하기
- 파일 삭제하기

Git의 관점

- 아무것도 모름

파일 저장과 Git 기록은 서로 다른 행동입니다

STAGING AREA

Staging 영역은 이번 배송에 넣을 물건을 고르는 포장대



```
$ git add A.txt C.txt
```

add는 저장이 아니라 "다음 기록에 포함할 변경 선택"입니다

Repository는 날짜별로 사진을 꽂아두는 공유 앨범



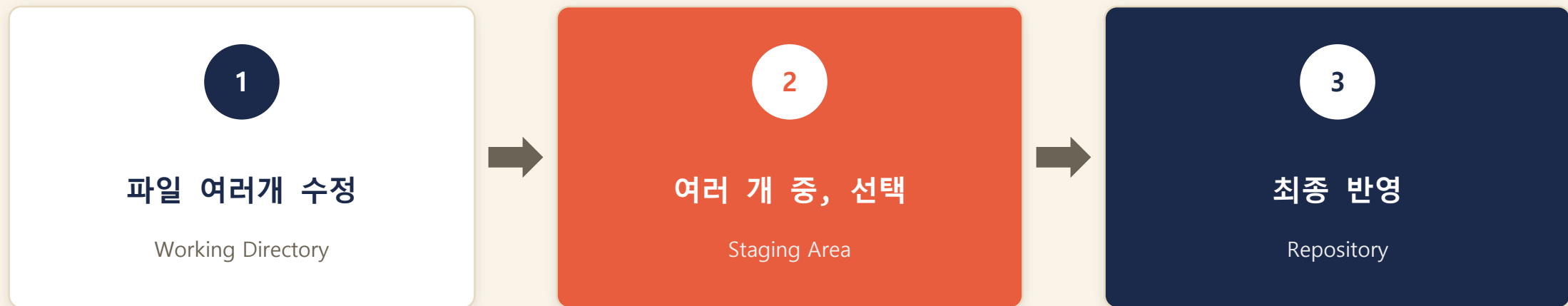
공유 앨범

선택한 변경 묶음 + 메시지 + 작성자 + 시간

- 최신 기록이 위에 쌓이는 시간순 앨범

FULL FLOW

작업 → 선택 → 기록의 흐름을 먼저 기억한다



코드 작업 → 코드 선별 → 코드 반영

INIT

새 저장소 시작

일반 작업 공간에 Git을 사용할 수 있도록 선언.

사용 시점

처음 한 번만 실행

일반 폴더



Git 관리 폴더

```
$ git init
```

▶ 실행 결과

```
Initialized empty Git repository in /project/.git/
```

STATUS

현재 상태 확인

어떤 파일이 바뀌었고 staging 되었는지 확인

사용 시점

작업 전후 가장 자주 사용

현재 작업



상태 목록

```
$ git status
```

▶ 실행 결과

```
On branch main
```

```
Changes not staged for commit:
```

```
  modified:   index.html
```

ADD

기록할 변경 선택

파일을 Working Directory에서 Staging Area로 이동

사용 시점

파일별 또는 전체 선택 가능

작업 공간



스테이징

```
$ git add [파일명]
```

```
$ git add .
```

▶ 실행 결과

(add는 출력이 없습니다 – 정상)

```
$ git status → Changes to be committed: index.html
```

DIFF

실제 변경 내용 확인

추가·삭제된 줄을 비교해 의도한 수정인지 검토

사용 시점

작업 중 또는 staging 후 비교

변경 전



변경 후

```
$ git diff [파일명]
```

▶ 실행 결과

```
- <p>Old text</p>  
+ <p>New text</p>
```

COMMIT

하나의 기록으로 확정

staging 된 변경을 메시지와 함께 저장합니다.

사용 시점

작고 의미 있는 단위로 기록

스테이징



저장소

```
$ git commit -m "로그인 기능 개발"
```

메시지만 읽어도 변경 이유가 보이게 작성하세요

▶ 실행 결과

[main 3f2a1c9] 로그인 오류 수정

1 file changed, 2 insertions(+), 1 deletion(-)

지난 기록 확인

commit ID·작성자·시간·메시지를 최신순으로 확인

사용 시점

기록을 빠르게 탐색

커밋 기록



시간순 목록

```
$ git log
```

▶ 실행 결과

```
3f2a1c9 (HEAD -> main) 로그인 오류 수정  
8b6e2d1 초기 커밋
```

reset과 revert는 "기록을 되돌림"

reset

일기장 찢어 버리기

현재 브랜치가 가리키는 위치를 과거 커밋으로 이동

```
$ git reset --soft HEAD~1
```

개인 로컬 기록 정리

revert

일기장에 취소선으로 지우기

기존 기록을 남기고 취소하는 새 커밋 생성

```
$ git revert <commit_id>
```

공유된 기록의 안전한 취소

현업에서는 reset은 사용 안함. Revert를 주로 사용!

이 세 가지만 피해도 절반은 성공

1



add 안 하고 commit

"커밋할 게 없다"는 메시지가 뜹니다.

해결: `git add` 로 먼저 스테이징

2



커밋 메시지 대충 쓰기

"수정", "asdf" 같은 메시지는 나중에 log를 봐도 무엇을 고쳤는지 알 수 없습니다.

해결: 무엇을 왜 바꿨는지 간결하게

3

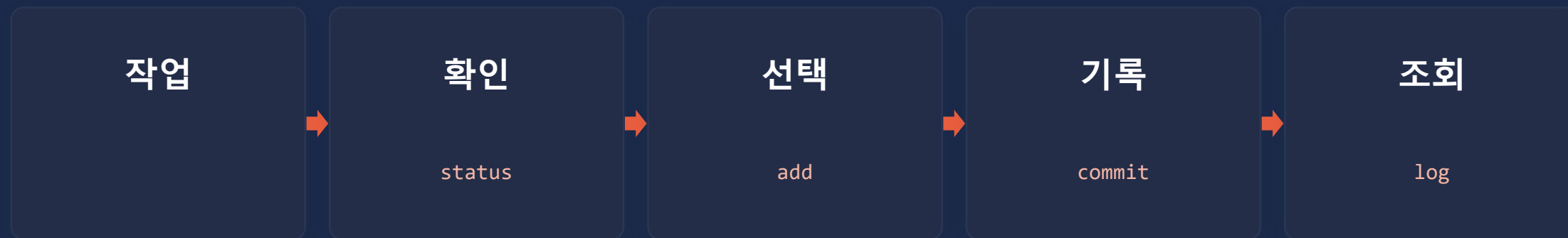


아무 파일 다 올리기

용량 큰 파일이나 API Key 같은 민감 정보가 그대로 push됩니다.

해결: 신중하게 선택하기

명령어와 이동 순서



복구

reset · revert는 실행 전 상태와 목적부터 확인하세요

다음 > 내 컴퓨터의 기록을 GitHub와 연결하고 브랜치로 협업하기

03

GitHub · 브랜치 · 협업

내 기록을 팀과 공유하는 원격 저장소 흐름으로 확장하기

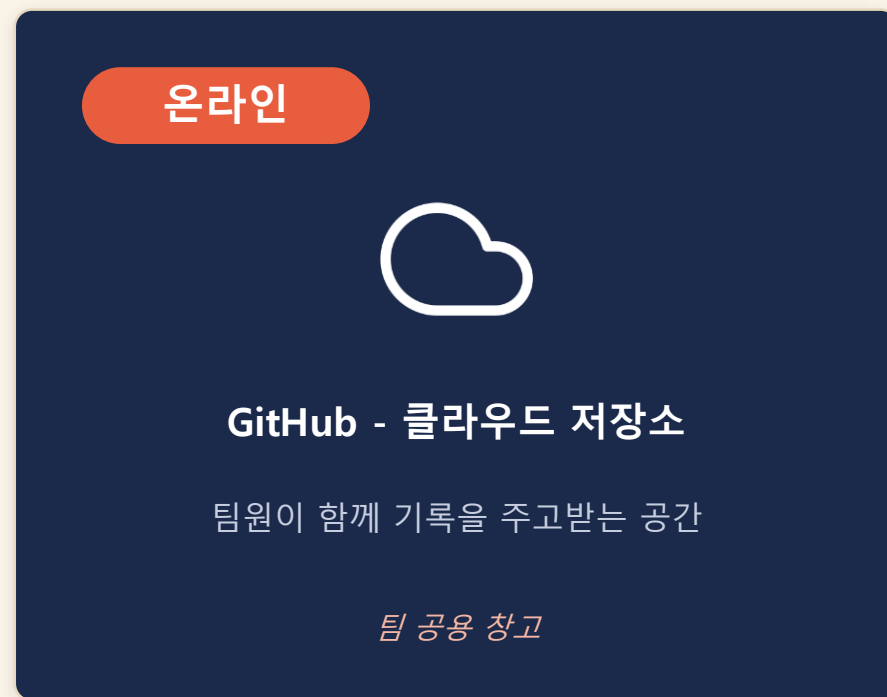
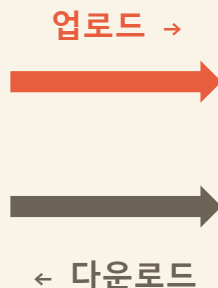
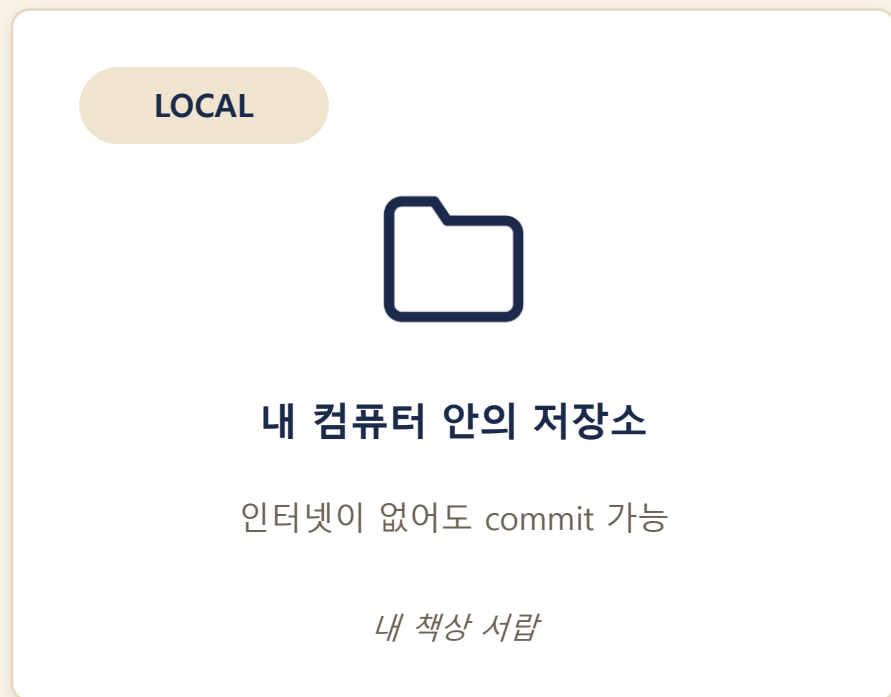
branch

merge

conflict

CONNECTION

Local은 내 노트북, Github는 클라우드



commit은 Local에 저장되고, 업로드를 해야 클라우드 환경(인터넷)에 공유됩니다

START OPTIONS

프로젝트 시작은 init 또는 clone 두 가지

git init

새 공책 시작

내 컴퓨터의 기존 폴더를 새 Git 저장소로 만들기

```
$ git init
```

새 프로젝트

git clone

공책 전체 복사

GitHub의 파일과 모든 커밋 기록을 내 컴퓨터로 복제

```
$ git clone <저장소 주소>
```

기존 원격 프로젝트 참여

PUSH

push는 내 기록(커밋)을 팀 공용 창고로 보내는 것



Local Repository

commit 완료



GitHub

팀원이 확인 가능

```
$ git push origin main
```

▶ 실행 결과

```
Enumerating objects: 5, done.
```

```
To https://github.com/gituser/git-study.git
```

```
8b6e2d1..3f2a1c9  main -> main
```

PULL

pull은 GitHub의 최신 기록을 내 컴퓨터로 가져오는 것

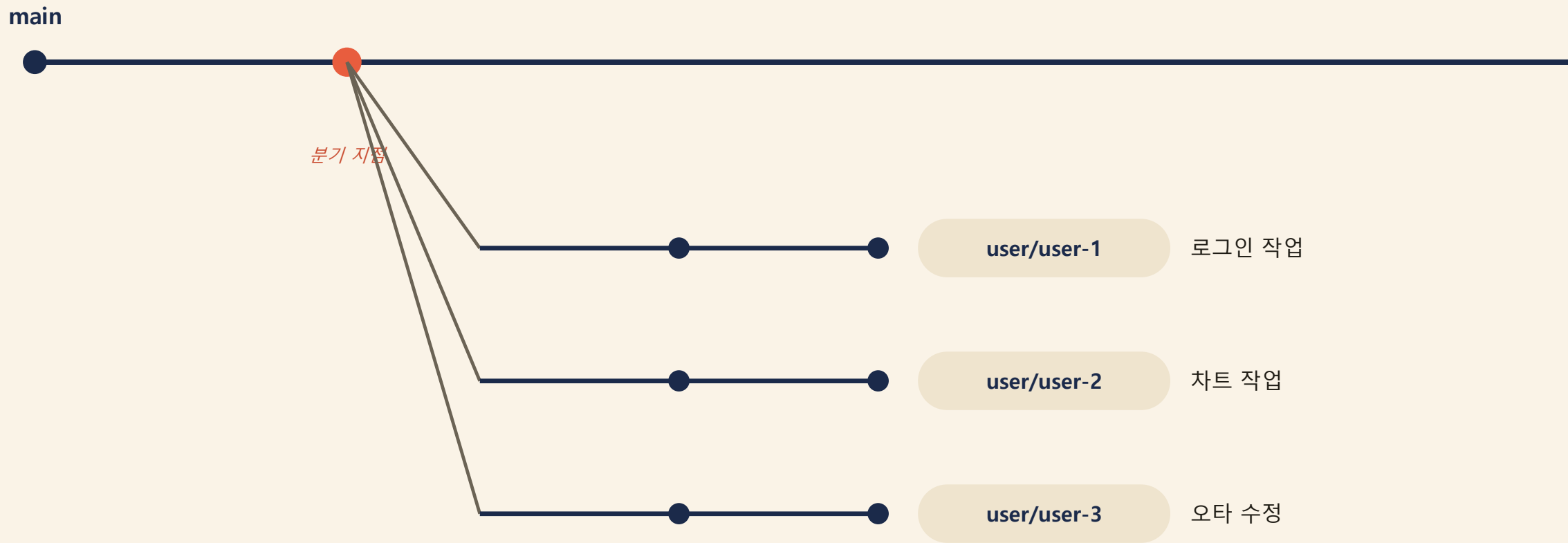


```
$ git pull
```

팀원이 push한 내용을 내 컴퓨터로 가져와, 지금 작업 중인 브랜치에 바로 반영합니다

BRANCH

Branch는 main을 복사해서 만든 각자의 작업 공간



main은 그대로 두고, 각자 브랜치에서 작업한 뒤 완성된 것만 main에 합칩니다

브랜치 생성과 이동은 checkout or switch로 구분

목록 확인

```
$ git branch
```

기존 브랜치로 이동

```
$ git checkout main
```

브랜치 생성

```
$ git branch [만들 브랜치명]
```

현재 위치 확인

```
$ git status
```

명령 실행 전, 지금 어느 브랜치인지 먼저 확인 필요

MERGE

merge는 완성된 사본의 변경을 원본에 합치는 과정



```
$ git checkout main  
$ git merge user/user-1
```

합칠 대상이 아니라 "합쳐 받을 브랜치"로 먼저 이동합니다

▶ 실행 결과

```
Updating 8b6e2d1..3f2a1c9 (Fast-forward)  
index.html | 4 ++--
```

CONFLICT

Conflict는 같은 문장을 서로 다르게 고친 상황

USER 1

"오늘 날씨는 춥다"

today.txt 같은 줄을 수정

main

"오늘 날씨는 차갑다"

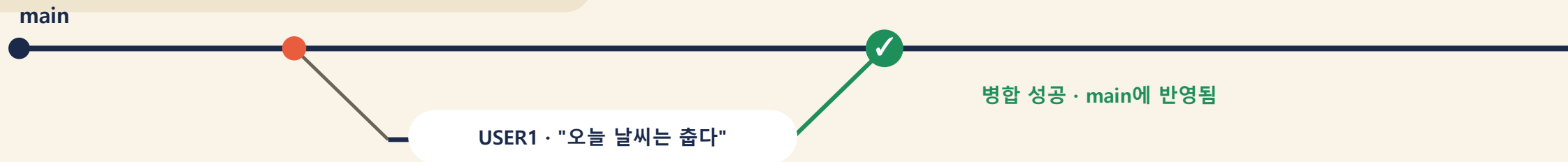
today.txt 같은 줄을 수정

Git은 누구의 선택이 정답인지 모름.

CONFLICT

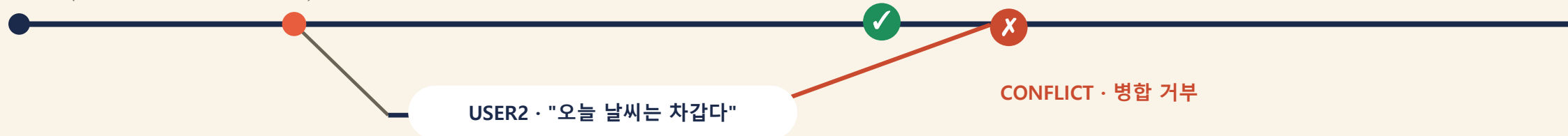
USER1이 merge한 뒤, USER2가 merge하면 벌어지는 일

STEP 1 · USER1이 먼저 merge



STEP 2 · USER2가 이후에 merge 시도

main (이미 "춥다"로 반영된 상태)



같은 부분을 서로 다르게 고쳤기 때문에, Git이 판단을 못 하고 사람에게 선택을 요청합니다

Conflict가 발생하면 파일 안에 이렇게 표시됩니다

weather.txt

<<<<<< HEAD

오늘 날씨는 춥다

=====

오늘 날씨는 차갑다

>>>>>> user2-branch

HEAD(main)의 내용과 feature/login의 내용이 마커 사이에
나란히 표시됩니다. 이 마커는 Git이 자동으로 남긴 표시입니다.

해결 방법

- 1 충돌 마커(<<<, ==, >>>) 사이 내용을 확인한다
- 2 남길 내용만 남기고 마커 3줄을 모두 삭제한다
- 3 git add 파일명 으로 해결을 표시한다
- 4 git commit 으로 충돌 해결을 확정한다

마커를 지우지 않으면 충돌 표시가 코드에 그대로 남습니다

RULES

충돌을 줄이는 가장 쉬운 방법은 작은 단위와 잦은 공유

1

작업 전

git pull로 최신화

2

작업 중

변경이 있을 때는, 바로 공유

3

역할 분담

가능하면 같은 파일 동시 수정 피하기

4

공유 전

Merge 대신 PR 사용하기

PR은 "이 변경, 합쳐도 될까요?"라고 묻는 과정

feature/login



Pull Request 생성



팀원 리뷰·댓글



Approve



main에 merge

바로 merge하지 않고 PR을 거치는 이유 — 코드 리뷰, 논의 기록, 실수 방지

왜 필요한가

팀원이 변경 내용을 미리 검토하고 의견을 남길 수 있어, merge 전에 문제를 걸러냅니다.

어디서 만드나

GitHub 저장소 상단의 Pull requests 탭에서 "New pull request"로 생성합니다.

Conflict가 있다면

PR 화면에서도 충돌 여부를 보여주며, 로컬에서 merge·해결 후 다시 push하면 됩니다.

GitHub에서 PR은 이런 화면으로 만들고 진행합니다

github.com/gituser/git-study/compare

① 생성

CodeIssuesPull requestsActionsSettings

Comparing changes

base: main <- compare: dk-branch

Able to merge — 충돌 없이 합칠 수 있습니다

Title

로그인 화면 기능 추가

Add a description

- 로그인 폼 UI 추가
- 입력값 검증 로직 반영

Create pull request

github.com/gituser/git-study/pull/12

② 리뷰

CodeIssuesPull requestsActionsSettings

로그인 화면 기능 추가Open

gituser wants to merge feature/login into main

B reviewer_b 님이 변경 사항을 승인했습니다Approved

reviewer_b: "예러 메시지 문구만 조금 다듬어주세요"

All checks have passed

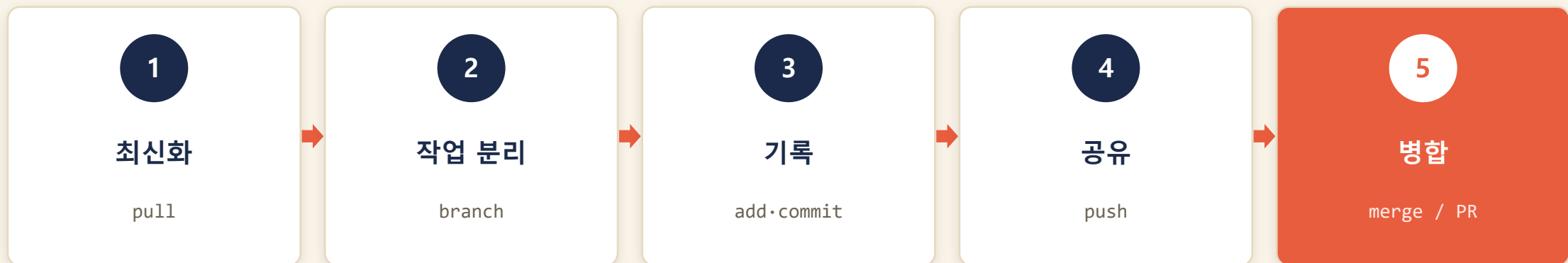
Merge pull requestmain에 반영 완료

SESSION 03 · GitHub·브랜치·협업

54

TEAM FLOW

팀 작업은 최신화 → 분리 → 기록 → 공유 → 병합



작업 시작 전 pull, 작업 종료 후 push를 습관화

한 브랜치에는 한 가지 목적의 변경만 담습니다

원격 저장소에 올라가면 안 되는 정보

절대 **commit**하지 않기

.env

*.pem

credentials.json

secrets/

비밀번호 · API Key · 인증서

· 클라우드 Credential

· 데이터베이스 접속 정보

· 개인정보·민감정보



.gitignore로 예방하고 commit 전 status로 확인

이미 공개했다면 삭제만 하지 말고, 키를 즉시 폐기·재발급

.gitignore는 애초에 택배 상자에 넣지 않을 목록

Git이 추적하지 않을 파일과 폴더를 지정

주로 제외하는 항목

- 비밀번호 · API Key · 환경 설정
- 빌드 결과물 · 대용량 임시 파일
- 개인 IDE 설정 · 로그 파일

```
# 비밀 정보  
.env
```

```
# 결과물·임시 파일  
dist/  
*.log
```

COMMIT MESSAGE

좋은 commit 메시지는 팀의 검색어

형식

타입: 간결한 변경 내용

feat

새 기능

feat: 로그인 화면 추가

fix

버그 수정

fix: 빈 값 입력 오류 수정

docs

문서 수정

docs: 설치 방법 추가

refactor

구조 개선

refactor: 중복 함수 분리

핵심은 기록을 안전하게 공유하는 순서

Local

commit으로 내 기록 만들기

Remote

push와 pull로 팀 기록 주고
받기

Branch

작업을 분리하고 merge로
합치기

Safety

status 확인과 작은 commit
을 습관화

다음 : 화면을 따라 직접 저장소 생성부터 협업까지 실습

05

언급하지 않았지만 알아두면 좋은 것들

추가 명령어 · 필수 파일 · 브랜치 전략 · AI 시대의 Git까지

stash

commit rule

branch 전략

README

AI

stash는 "지금 작업을 잠깐 서랍에 넣어두기"



이런 상황에 씁니다

작업 중인데 아직 commit하기엔 애매하고, 그런데 브랜치는 지금 바꿔야 할 때

- 1 임시로 작업 내용을 서랍(stash)에 넣어둔다
- 2 깨끗해진 상태에서 다른 브랜치로 이동
- 3 나중에 돌아와 다시 꺼내 이어서 작업

```
$ git stash
$ git switch other-branch
... 작업 후 다시 돌아와서
$ git switch feature/login
$ git stash pop
```

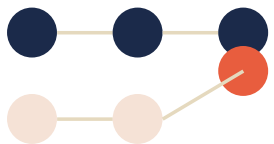
자주 쓰는 명령어

<code>git stash</code>	임시 저장
<code>git stash list</code>	저장 목록 확인
<code>git stash pop</code>	꺼내면서 삭제

rebase는 히스토리를 다시 쓰는 방식입니다

merge

지금까지 배운 방식



두 흐름을 그대로 두고 하나의 병합 커밋으로 합칩니다 — 기존 히스토리 보존

rebase



브랜치의 시작점을 옮겨 하나의 일직선 히스토리로 만듭니다 — 깔끔하지만 기록을 다시 씀

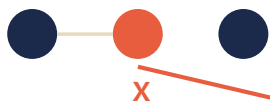
원칙: 이미 push해서 팀원과 공유된 커밋은 rebase 하지 않습니다

💡 "merge와 rebase의 차이"는 기술 면접에서 자주 나오는 질문입니다 — 개념만 기억해두세요

CHERRY-PICK

다른 브랜치의 커밋 하나만 꼭 집어오기

hotfix 브랜치



cherry-pick

main 브랜치



```
$ git cherry-pick <commit_id>
```

언제 쓰나

급한 버그 수정이 다른 브랜치에만 있는데, 그 커밋 하나만 지금 브랜치에 필요할 때

merge와 다른 점

브랜치 전체가 아니라 커밋 하나만 선택적으로 가져옵니다

git tag로 배포 버전에 이름을 남깁니다



특정 커밋에 버전 이름표를 붙입니다

- 1 배포한 시점의 커밋을 나중에도 정확히 찾을 수 있음
- 2 릴리즈 노트·변경 이력 문서와 함께 자주 사용
- 3 v1.0.0 처럼 버전 규칙(semantic versioning)을 따르는 것이 일반적

```
$ git tag v1.0.0  
$ git push origin v1.0.0
```

버전 이름 예시

v1.0.0 정식 출시

v1.1.0 기능 추가

v1.1.1 버그 수정

팀 커밋 메시지 규칙을 이렇게 정해보세요

형식

팀원들끼리 규칙을 만들어 사용

feat

새 기능 추가

feat(login): 카카오 로그인 추가

fix

버그 수정

fix(cart): 수량 0일 때 오류 수정

docs

문서 수정

docs(readme): 설치 방법 보완

style

포매팅, 세미콜론 등

style: 들여쓰기 정리

refactor

기능 변화 없는 구조 개선

refactor(api): 중복 함수 통합

test

테스트 코드

test(user): 회원가입 테스트 추가

chore

빌드 설정 등 기타 작업

chore: 패키지 버전 업데이트

좋은 예 fix(auth): 만료된 토큰 재발급 처리 추가

나쁜 예 수정함, 버그 고침, asdf

README.md는 저장소의 첫인상입니다



저장소를 열면 자동으로 보이는 설명서

- 1 프로젝트가 무엇을 하는지 (소개)
- 2 어떻게 설치하고 실행하는지
- 3 어떻게 사용하는지 (예시)
- 4 어떤 기술을 사용했는지

포트폴리오에서는 면접관이 가장 먼저 보는 파일입니다

README.md

Git Study

소개

초보자를 위한 Git 학습 자료

설치

git clone [주소]

사용법

git status 로 시작하세요

이 프로젝트가 필요로 하는 재료 목록



Python 프로젝트의 재료(라이브러리) 목록

레시피의 재료 목록처럼, 이 프로젝트를 실행하려면 어떤 라이브러리가 필요한지 적어둔 파일입니다

- 다른 사람의 컴퓨터에서도 똑같이 실행되도록 보장
- requirements.txt가 없으면 어떤 라이브러리가 필요한지 알 수 없음

requirements.txt도 반드시 Git으로 함께 관리하세요

```
requirements.txt
pandas==2.1.0
numpy==1.26.0
requests>=2.31.0
```

자주 쓰는 명령어

```
pip freeze > requirements.txt
```

현재 설치된 목록 저장

```
pip install -r requirements.txt
```

목록대로 한 번에 설치

이름만 봐도 무슨 작업인지 알 수 있게

feature/

새 기능 추가

feature/login-form

fix/

일반 버그 수정

fix/button-color

hotfix/

운영 중 긴급 수정

hotfix/payment-error

release/

배포 준비

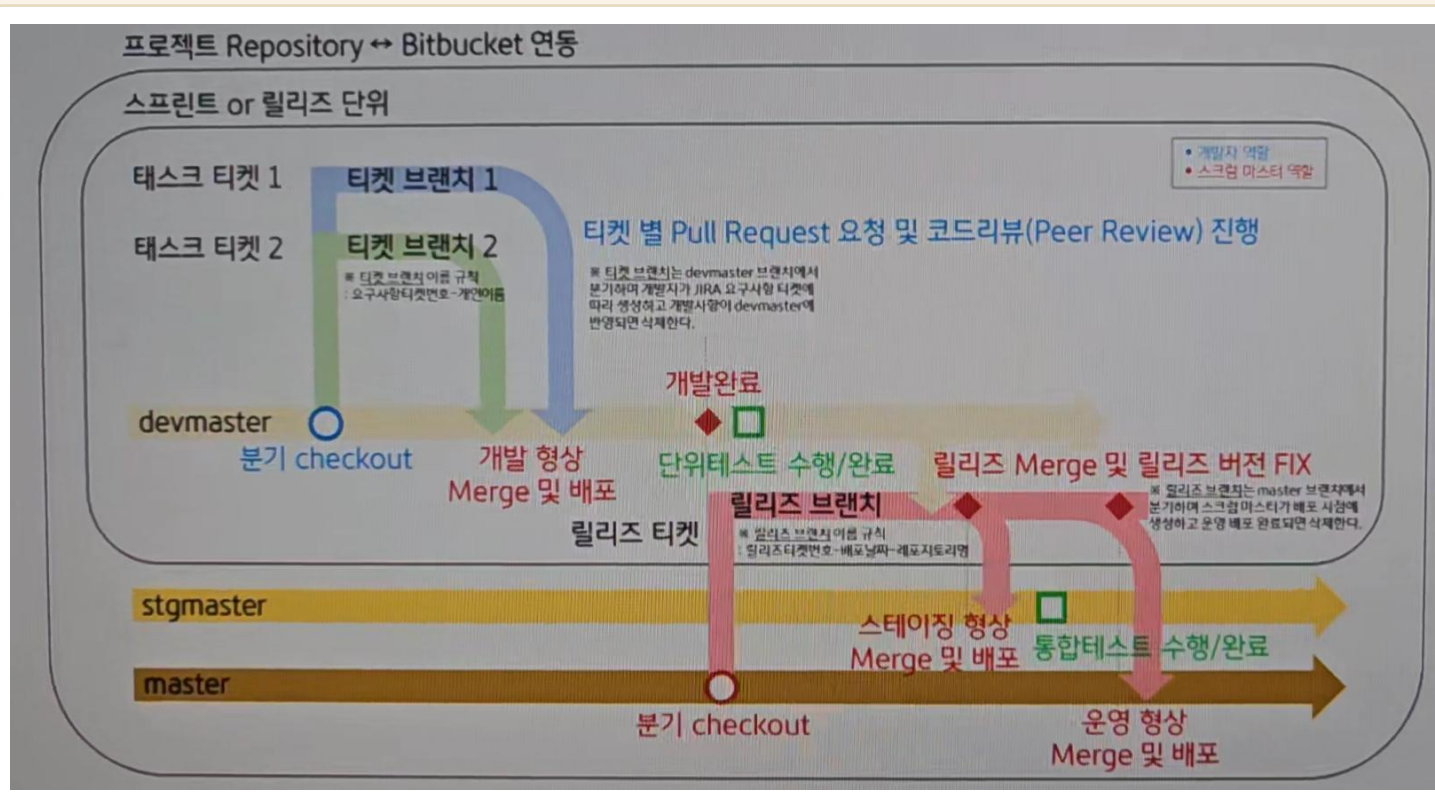
release/v1.2.0

docs/

문서 작업

docs/readme-update

실제로 이런 브랜치 전략을 사용하고 있습니다



이 자료가 보여주는 것

- 1 티켓마다 브랜치를 따로 만들어 devmaster에서 PR·리뷰 후 병합
- 2 릴리즈 시점에 release 브랜치를 분기해 stgmaster(스테이징)로 먼저 배포
- 3 검증이 끝나면 같은 release 브랜치를 master(운영)에 최종 배포
- 4 개발자와 스크럼마스터의 역할을 브랜치 단위로 명확히 구분

실제 협업에서 사용하고 있는 자료를 그대로 가져왔습니다

AI가 바꾼 Git 사용법

AI 이전

AI 이후

커밋 메시지 작성

변경 내용을 하나하나 요약해서 타이핑

➡ diff를 읽고 AI가 커밋 메시지 초안을 제안

PR 설명 작성

무엇을 왜 바꿨는지 직접 정리해서 작성

➡ AI가 변경 내용을 요약해 PR 설명을 자동 생성

코드 리뷰

리뷰어가 한 줄씩 읽으며 문제를 발견

➡ AI가 먼저 잠재적 버그·개선점을 짚어줌

Conflict 해결

마커를 보며 직접 판단해서 수정

➡ 에디터의 AI가 두 내용을 보고 해결안을 제안

AI는 초안을 제안할 뿐 — 최종 확인과 판단은 여전히 사람의 몫입니다

AI가 이 모든 걸 대신하게 하려면 — 규칙을 MD 파일로 알려주세요



AI 코딩 도구는 규칙 파일을 읽습니다

Claude Code · Cursor · Copilot 같은 도구는 저장소 루트의 MD 파일을 자동으로 읽고, 그 규칙에 맞춰 작업합니다

도구별 파일 이름

CLAUDE.md

AGENTS.md

.cursorrules

MD 파일에 담을 내용

- 1 커밋 메시지 규칙 (type-scope 형식)
- 2 브랜치 네이밍 규칙과 전략
- 3 PR 작성 가이드
- 4 자주 쓰는 명령어와 금지 사항

CLAUDE.md

Commit Convention

type(scope): subject
feat / fix / docs / refactor

Branch Strategy

feature/*, fix/*, hotfix/*
devmaster -> stgmaster -> master

Commands

Always run tests before commit
Never force-push to main

한 번 작성해두면, 매번 설명하지 않아도 AI가 팀 규칙대로 커밋·브랜치·PR을 제안합니다

오늘 다룬 것들을 실무에서 마주치면 이렇게

명령어

stash, rebase-cherry-pick-tag는
개념 위주로 기억

커밋·브랜치 규칙

팀이 합의한 커밋 컨벤션과 브
랜치 전략을 문서로 남기기

필수 파일

README.md와 requirements.txt
는 프로젝트마다 기본으로 작성

AI 활용

MD 파일로 규칙을 알려주면, AI
가 그 규칙대로 제안 — 최종 판
단은 사람의 몫